Below is a **practical "day-to-day Java master list"** of the **most commonly used classes + methods** across **Java SE versions (Java 1.0 → Java 21)**, with **real-time use cases** and **mini examples**.

✅ Focus: **classes/methods you will actually use in real software development** (backend, microservices, data processing, utilities, concurrency, file handling, REST clients, etc.)
❌ Not included: extremely rare / academic / legacy AWT-only APIs

---

1) Core Language + java.lang (Most used daily)

Area	Class	Common Methods	Real Use Case	Example (mini)
Strings	String	length(), isEmpty(), isBlank(), substring(), contains(), startsWith(), endsWith(), replace(), split(), trim(), strip()	Handling request params, formatting, validations	"abc".contains("a")
Mutable String	StringBuilder	append(), insert(), delete(), reverse(), toString()	Building logs, SQL, JSON	new StringBuilder().append("Hi").append(name)
Mutable thread-safe	StringBuffer	append(), toString()	Legacy thread safe string operations	new StringBuffer("x").append("y")
Objects base	Object	toString(), equals(), hashCode(), getClass()	Entity equality, debugging	obj.toString()
Wrapper types	Integer, Long, Double, Boolean	parseInt(), valueOf(), compareTo(), toString()	parsing input values	Integer.parseInt("100")
System	System	currentTimeMillis(), nanoTime(), getenv(), getProperty(), exit()	benchmark, environment values	System.getenv("PATH")
Runtime	Runtime	getRuntime(), availableProcessors()	CPU info, memory debugging	Runtime.getRuntime().availableProcessors()
Exceptions	Throwable, Exception, RuntimeException	getMessage(), printStackTrace()	error handling	throw new IllegalArgumentException("bad")

---

2) java.util Collections (Everyday backend coding)

Category	Class	Common Methods	Use Case	Example
Dynamic list	ArrayList	add(), get(), set(), remove(), size()	store records in order	list.add(user)
Linked list	LinkedList	addFirst(), addLast(), poll()	queue + stack	queue.poll()
Set unique	HashSet	add(), contains(), remove()	unique emails, IDs	set.contains(id)
Ordered set	TreeSet	add(), first(), last()	sorted unique list	treeSet.first()
Map key-value	HashMap	put(), get(), getOrDefault(), containsKey(), remove()	caching, config map	map.getOrDefault(k,0)
Ordered map	LinkedHashMap	put(), iteration order	LRU cache base	insertion order
Sorted map	TreeMap	firstKey(), lastKey(), subMap()	leaderboard ranking	tm.subMap(a,b)
Optional	Optional	ofNullable(), isPresent(), orElse(), orElseThrow(), map(), ifPresent()	avoiding null checks	Optional.ofNullable(x).orElse("NA")
Utilities	Collections	sort(), reverse(), unmodifiableList()	list ops	Collections.sort(list)
Arrays helper	Arrays	asList(), sort(), binarySearch(), copyOf()	array operations	Arrays.sort(arr)
Objects helper	Objects	requireNonNull(), equals(), hash()	validation	Objects.requireNonNull(user)

---

3) java.util.stream (Java 8+ daily functional style)

Stream Item	Class/Type	Key Methods	Use Case	Example
Stream creation	Stream	of(), empty(), generate(), iterate()	pipeline processing	Stream.of(1,2,3)
Filter/map	Stream	filter(), map(), flatMap()	DTO transform	users.stream().map(User::getEmail)
Terminal ops	Stream	collect(), forEach(), reduce(), count(),	aggregation	.collect(Collectors.toList())

		anyMatch()		
Collectors	Collectors	toList(), toSet(), toMap(), groupingBy(), joining()	reports & grouping	groupingBy(User::getCity)
Primitive streams	IntStream, LongStream	range(), sum(), average()	numeric computation	IntStream.range(1,10).sum()

---

4) Date/Time API (Java 8+ professional backend)

Area	Class	Methods	Use Case	Example
Current time	Instant	now(), toEpochMilli()	timestamps in logs/db	Instant.now().toEpochMilli()
Local date	LocalDate	now(), plusDays(), minusDays(), parse()	birthdays, due date	LocalDate.parse("2026-01-17")
Local time	LocalTime	now(), plusHours()	time schedules	LocalTime.now()
DateTime	LocalDateTime	now(), format()	event times	LocalDateTime.now()
Zoned time	ZonedDateTime	now(zone)	global timezone apps	ZonedDateTime.now(ZoneId.of("Asia/Kolkata"))
Duration	Duration	between(), toMillis()	latency measurement	Duration.between(a,b)
Period	Period	between(), getDays()	HR tenure	Period.between(d1,d2)
Formatter	DateTimeFormatter	ofPattern(), format()	display formatting	dt.format(DateTimeFormatter.ISO_DATE_TIME)

---

5) java.io + java.nio.file (File handling in real apps)

Category	Class	Methods	Use Case	Example
File path	Path	of(), resolve(), getFileName()	safe file path operations	Path.of("data","a.txt")
Files util	Files	readString(), writeString(), readAllLines(), copy(), move(), exists()	file read/write	Files.readString(path)
Legacy file	File	exists(), mkdirs(), listFiles()	older systems	new File("logs").mkdirs()
Buffered reading	BufferedReader	readLine(), lines()	reading large files	br.lines().forEach(...)
Buffered writing	BufferedWriter	write(), newLine()	writing report output	bw.write(line)
Serializatio	ObjectOut	writeObject()	caching	oos.writeObject(obj)

n	putStream		objects	

---

6) Networking + HTTP client (modern Java 11+)

Area	Class	Methods	Use Case	Example
URL	URL	openStream()	legacy HTTP calls	new URL(url).openStream()
HTTP client	HttpClient	newHttpClient(), send(), sendAsync()	REST API integration	client.send(req, BodyHandlers.ofString())
Request	HttpRequest	newBuilder(), GET(), POST()	build request	HttpRequest.newBuilder(URI.create(...))
Response	HttpResponse	statusCode(), body()	handle REST response	res.body()

---

7) Concurrency (Java threads → executors → virtual threads)

Category	Class	Methods	Use Case	Example
Basic thread	Thread	start(), sleep(), join()	low-level tasks	new Thread(r).start()
Runnable	Runnable	run()	thread job	() -> process()
Executor	ExecutorService	submit(), execute(), shutdown()	backend parallel jobs	pool.submit(task)
Executors	Executors	newFixedThreadPool(), newCachedThreadPool()	creating pools	Executors.newFixedThreadPool(10)
Future	Future	get(), cancel()	async result	future.get()
Scheduled	ScheduledExecutorService	scheduleAtFixedRate()	cron-like jobs	schedule polling
CompletableFuture	CompletableFuture	supplyAsync(), thenApply(), thenCompose(), allOf()	async REST calls	CompletableFuture.allOf(f1,f2)
Locks	ReentrantLock	lock(), unlock(), tryLock()	critical section	lock.lock()
Atomic	AtomicInteger	incrementAndGet(), get()	thread-safe counter	count.incrementAndGet()
Concurrent map	ConcurrentHashMap	computeIfAbsent(), putIfAbsent()	caches, sessions	map.computeIfAbsent(k,...)
Virtual threads (Java 21)	Executors.newVirtualThreadPerTaskExecutor()	submit()	massive concurrency	best for high I/O apps

---

8) Logging + Diagnostics (enterprise daily)

Area	Class	Methods	Use Case	Example
JDK logger	java.util.logging.Logger	info(), warning(), severe()	simple apps	Logger.getLogger("x").info("done")
Stack trace	StackTraceElement	getMethodName() etc	debugging	exception traces

In real microservices, you typically use **SLF4J + Logback**, but in pure Java this is your base.

---

9) Regular Expressions (validation & parsing)

Class	Methods	Use Case	Example
Pattern	compile(), matcher()	compile regex	Pattern.compile("\\d+")
Matcher	find(), group(), matches()	extract emails/IDs	m.find()

---

10) Java 8+ functional programming utilities

Type	Interface	Key Methods	Use Case	Example
Function	Function<T,R>	apply()	mapping objects	x -> x.toUpperCase()
Predicate	Predicate<T>	test(), and(), or()	filtering	s -> s.length()>3
Consumer	Consumer<T>	accept()	logging/printing	System.out::println
Supplier	Supplier<T>	get()	lazy init	() -> new ArrayList<>()

---

11) JDBC (still very common in real projects)

Class	Common Methods	Use Case	Example
DriverManager	getConnection()	open DB connection	DriverManager.getConnection(url,u,p)
Connection	prepareStatement(), setAutoCommit(), commit(), rollback()	transaction	conn.commit()
PreparedStatement	setString(), setInt(), executeQuery(), executeUpdate()	safe SQL	prevent SQL injection
ResultSet	next(), getString(), getInt()	read rows	while(rs.next())...

---

12) Java 17+ / 21 features used in modern codebases

Feature	Class/Keyword	Use Case	Example
Records (Java 16+)	record	immutable DTO	record User(String name,int age){}
Pattern matching	instanceof	simplify casts	if(obj instanceof String s)
Switch expressions	switch	clean branching	var x = switch(n){...};
Sealed classes	sealed	controlled inheritance	sealed interface Shape permits Circle {}
Virtual threads	Thread.ofVirtual() / executor	high concurrency	best for high I/O

✅ **Real-world Examples (Most common patterns)**

Example 1: Validate + group users by city (Collections + Stream)

```
Map<String, List<User>> usersByCity =
    users.stream()
        .filter(u -> u.getEmail() != null && !u.getEmail().isBlank())
        .collect(Collectors.groupingBy(User::getCity));
```

Example 2: Read file lines + store unique emails (NIO + Set)

```
Set<String> emails = new HashSet<>(Files.readAllLines(Path.of("emails.txt")));
```

Example 3: Call REST API (Java 11 HttpClient)

```
HttpClient client = HttpClient.newHttpClient();

HttpRequest req = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users"))
    .GET()
    .build();

HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode());
System.out.println(res.body());
```

Example 4: Async parallel tasks (CompletableFuture)

```
CompletableFuture<String> f1 = CompletableFuture.supplyAsync(() -> serviceA());
CompletableFuture<String> f2 = CompletableFuture.supplyAsync(() -> serviceB());

CompletableFuture.allOf(f1, f2).join();
```

If you want, I can generate a complete "Java Day-to-Day API Cheat Sheet" PDF

✅ With all classes grouped by category + signature style
✅ Java 8 / 11 / 17 / 21 differences
✅ common interview questions + production use cases

Just say: **"Create PDF cheat sheet"**.

Perfect ✅
Below is the **Day-to-Day Java API Cheat Sheet** with:

✅ **All classes grouped by category**
✅ **Signature style (method signatures like in IDE / docs)**
✅ **Real-time usage meaning (why you use it in projects)**

✅ JAVA DAY-TO-DAY CLASSES (Grouped + Signature Style)

---

1) java.lang (Core language – used everywhere)

✅ String

Signature	Meaning / Use
int length()	length check
boolean isEmpty()	empty validation
boolean isBlank() ***(Java 11+)***	blank/whitespace validation
String substring(int beginIndex)	extract part
String substring(int beginIndex, int endIndex)	extract range
boolean contains(CharSequence s)	search
boolean startsWith(String prefix)	prefix validation
boolean endsWith(String suffix)	suffix validation
String replace(CharSequence target, CharSequence replacement)	replace in text
String[] split(String regex)	tokenization
String trim()	remove spaces
String strip() ***(Java 11+)***	Unicode-safe trim

✅ StringBuilder

Signature	Use
StringBuilder append(String str)	build messages/logs/SQL
StringBuilder insert(int offset, String str)	insert in middle

StringBuilder delete(int start, int end)	remove substring
String toString()	final string

✅ Integer, Long, Double

Class	Signatures
Integer	static int parseInt(String s)
	static Integer valueOf(String s)
	int compareTo(Integer another)
Long	static long parseLong(String s)
Double	static double parseDouble(String s)

✅ System

Signature	Use
static long currentTimeMillis()	timestamp
static long nanoTime()	benchmarking
static String getenv(String name)	env variables
static String getProperty(String key)	JVM properties
static void exit(int status)	stop JVM

✅ Objects

Signature	Use
static <T> T requireNonNull(T obj)	mandatory validation
static boolean equals(Object a, Object b)	null-safe equals
static int hash(Object... values)	hashcode generation

---

2) java.util Collections (Most important daily)

✅ List

ArrayList<E>

Signature	Use
boolean add(E e)	insert data
E get(int index)	fetch
E set(int index, E element)	update
E remove(int index)	delete
int size()	count

LinkedList<E>

Signature	Use
void addFirst(E e)	queue/stack
void addLast(E e)	queue
E poll()	remove head

✅ Set

HashSet<E>

Signature	Use
boolean add(E e)	unique store
boolean contains(Object o)	check duplicates
boolean remove(Object o)	remove

TreeSet<E>

Signature	Use
E first()	smallest element
E last()	largest element

✅ Map

HashMap<K,V>

Signature	Use
V put(K key, V value)	insert
V get(Object key)	fetch
V getOrDefault(Object key, V defaultValue)	safe fetch
boolean containsKey(Object key)	check
V remove(Object key)	delete
Set<Map.Entry<K,V>> entrySet()	iterate

TreeMap<K,V>

Signature	Use
K firstKey()	min key
K lastKey()	max key
NavigableMap<K,V> subMap(...)	range filtering

✅ Optional<T> (Java 8+)

Signature	Use
static <T> Optional<T> ofNullable(T value)	allow null
T orElse(T other)	fallback
T orElseThrow()	force validation

<U> Optional<U> map(Function<? super T,? extends U> mapper)	transform

---

✅ Collections & Arrays

Class	Signatures	Use
Collections	static <T> void sort(List<T> list)	sorting
	static <T> List<T> unmodifiableList(List<? extends T> list)	read-only list
Arrays	static <T> List<T> asList(T... a)	array→list
	static void sort(int[] a)	array sort
	static int binarySearch(int[] a, int key)	fast search

---

3) Streams API (java.util.stream) Java 8+

✅ Stream<T>

Signature	Use
Stream<T> filter(Predicate<? super T> predicate)	filtering
<R> Stream<R> map(Function<? super T,? extends R> mapper)	transform
Stream<T> distinct()	remove duplicates
Stream<T> sorted()	sort
void forEach(Consumer<? super T> action)	side effects
long count()	count
Optional<T> findFirst()	first element
<R,A> R collect(Collector<? super T,A,R> collector)	convert to list/map

✅ Collectors

Signature	Use
static <T> Collector<T,?,List<T>> toList()	stream → list
static <T> Collector<T,?,Set<T>> toSet()	stream → set
static <T,K> Collector<T,?,Map<K,List<T>>> groupingBy(Function<? super T,? extends K> classifier)	group-by

---

4) Date & Time (java.time) Java 8+

Class	Signature	Use
LocalDate	static LocalDate now()	only date
	static LocalDate parse(CharSequence text)	parse date
	LocalDate plusDays(long daysToAdd)	add days
LocalDateTime	static LocalDateTime now()	date+time
Instant	static Instant now()	UTC timestamp
Duration	static Duration between(Temporal start, Temporal end)	time difference

DateTimeFormatter	static DateTimeFormatter ofPattern(String pattern)	formatting

---

5) File Handling (Modern) java.nio.file

✅ Path

Signature	Use
static Path of(String first, String... more)	build file path
Path resolve(String other)	join paths
Path getFileName()	filename

✅ Files

Signature	Use
static boolean exists(Path path)	check file
static String readString(Path path) ***(Java 11+)***	read full file
static Path writeString(Path path, CharSequence csq) ***(Java 11+)***	write
static List<String> readAllLines(Path path)	read lines
static Stream<String> lines(Path path)	huge file streaming
static Path copy(Path source, Path target, CopyOption... options)	copy
static Path move(Path source, Path target, CopyOption... options)	move

---

6) File Handling (Legacy) java.io

Class	Signature	Use
File	boolean exists()	old-style file ops
BufferedReader	String readLine()	read big text
BufferedWriter	void write(String str)	write
FileReader	constructor	old file reading
FileWriter	constructor	old file writing

---

7) Networking + REST APIs (Java 11+)

✅ HttpClient

Signature	Use
static HttpClient newHttpClient()	create client
<T> HttpResponse<T> send(HttpRequest request, BodyHandler<T> responseBodyHandler)	sync REST call
<T> CompletableFuture<HttpResponse<T>> sendAsync(...)	async REST

✅ HttpRequest

Signature	Use
static HttpRequest.Builder newBuilder(URI uri)	build request
Builder GET()	GET
Builder POST(BodyPublisher bodyPublisher)	POST
Builder header(String name, String value)	headers

✅ HttpResponse

Signature	Use
int statusCode()	HTTP code
T body()	response body

---

8) Concurrency (Threading in real software)

✅ Basic: Thread, Runnable

Class	Signature	Use
Thread	void start()	start thread
	static void sleep(long millis)	delay
Runnable	void run()	thread job

✅ Executor Framework (Recommended)

Class	Signature	Use
ExecutorService	Future<?> submit(Runnable task)	async task
	<T> Future<T> submit(Callable<T> task)	returns result
	void shutdown()	graceful shutdown
Executors	static ExecutorService newFixedThreadPool(int nThreads)	pool
ScheduledExecutorService	ScheduledFuture<?> scheduleAtFixedRate(...)	periodic task

---

✅ CompletableFuture (Async microservices)

Signature	Use
static <U> CompletableFuture<U> supplyAsync(Supplier<U> supplier)	async return
<U> CompletableFuture<U> thenApply(Function<? super T,? extends U> fn)	transform
CompletableFuture<Void> thenAccept(Consumer<? super T> action)	consume
static CompletableFuture<Void> allOf(...)	join many

✅ **Concurrent Collections**

Class	Signatures	Use
ConcurrentHashMap	V putIfAbsent(K key, V value)	safe cache
	V computeIfAbsent(K key, Function<? super K,? extends V> mappingFunction)	lazy load
AtomicInteger	int incrementAndGet()	counter

---

9) Regular Expressions (java.util.regex)

Class	Signature	Use
Pattern	static Pattern compile(String regex)	compile regex
Matcher	boolean matches()	full match
	boolean find()	find substring
	String group()	extract

---

10) Functional Interfaces (java.util.function) Java 8+

Interface	Signature	Use
Function<T,R>	R apply(T t)	mapping
Predicate<T>	boolean test(T t)	filtering
Consumer<T>	void accept(T t)	printing/logging
Supplier<T>	T get()	lazy init

---

11) JDBC Database API (Real-time enterprise)

Class	Signature	Use
DriverManager	static Connection getConnection(String url, String user, String password)	connect DB
Connection	PreparedStatement prepareStatement(String sql)	execute SQL
	void commit()	commit
	void rollback()	rollback
PreparedStatement	void setString(int parameterIndex, String x)	bind values
	ResultSet executeQuery()	select
	int executeUpdate()	insert/update/delete
ResultSet	boolean next()	rows
	String getString(String columnLabel)	value

---

12) Modern Java Features Classes (Java 17/21 daily)

Feature	Class/Keyword	Use Case
Records	record	DTO models

Sealed classes	sealed	controlled inheritance
Pattern matching	instanceof	safe casting
Virtual threads	Thread.ofVirtual() / Executors.newVirtualThreadPerTaskExecutor()	handle huge I/O concurrency

---

✅ **If you want I can deliver this in a printable format**

I can generate a **PDF + DOCX + Excel table** containing:

✅ Category-wise classes
✅ Signature style
✅ Real-time use case
✅ Example snippet

Just say: **"Generate PDF + Excel"**